Live Fast, Crash Hard: An Empirical Study of Bug Lifespans in ML-Integrated Systems

Mushfiqur Rahman
University of Dhaka
Dhaka, Bangladesh
bsse1130@iit.du.ac.bd

Abu Jafar Saifullah
University of Dhaka
Dhaka, Bangladesh
bsse1109@iit.du.ac.bd

Md Nurul Ahad Tawhid
University of Dhaka
Dhaka, Bangladesh
tawhid@iit.du.ac.bd

***Abstract*—Integrating machine learning (ML) into traditional software introduces unique reliability challenges. In this paper, we empirically examine the density of defects and the survival time of defects for 13 open-source software projects that integrate machine learning. Using the SZZ algorithm, we identify bug-introducing commits for machine learning code and non-machine learning code. Our results demonstrate a counter-intuitive finding: whereas machine learning code is significantly more defect-prone than traditional software code, the time it takes to fix bugs in machine learning code is almost twice as fast, at a median of 41.34 days, compared to traditional software code, which takes a median of 80.92 days. However, there is a significant percentage, at 40%, of defects that persist for more than 90 days.**

***Index Terms*—Machine Learning Systems, Empirical Software Engineering, Bug Survival Time, Defect Density, Mining Software Repositories.**

I. Introduction

Machine learning (ML) has become an integral component of many software systems, and the combination of data pipelines, ML training, and statistical inference with conventional software systems creates unique defect risk profiles [1]. Open-source ML software systems are composed of ML-related and non-ML-related components, and the key questions are whether ML-related components are more defect-prone and whether ML-related bugs are fixed later than non-ML-related bugs. Previous studies have proposed fault taxonomies and root causes of ML software systems [1], [2]. Software engineering research has extensively employed the SZZ model, which was proposed by Sliwerski et al. [3], to identify bug-fixing and bug-introducing commits, and the model has become a widely accepted approach to characterizing defect behavior; however, the defect behavior of ML-related and non-ML-related components within the same software systems remains unknown.

Our results demonstrate that the defect dynamics of ML-related code and non-ML code differ within the same code repositories. Although the percentage of ML-related code is lower, it still makes up a significant percentage of bug-introducing commits. Moreover, the risk patterns of ML-related code differ from non-ML code. In addition, the bug survival patterns for the two types of code differ, with more rapid resolution of bugs during the early stages for ML-related code.

Through the use of repository mining, SZZ, and ML-file classification, this study presents a comparative analysis of defect introduction and bug survival in OSS ML software products. In addition to presenting empirical evidence, this study also presents a dataset of bug introduction commits, which could be useful in replicating this work, benchmarking tools, and building tools in the area of ML software quality assurance.

In this paper, we conduct an empirical study to investigate the origin and characteristics of bugs in ML-integrated software products. Our study uses the curated dataset introduced in *Beyond the Model: An Empirical Study of Machine Learning Software Products*, which contains 262 open-source ML products that combine ML and traditional software components. We mine GitHub repositories to identify bug-fixing commits using issue references and commit message heuristics. We then apply the SZZ algorithm [3] to identify bug-introducing commits (BICs). For each BIC, we classify the modified files as ML-related or non-ML based on the imported libraries (e.g., TensorFlow, PyTorch, Scikit-learn, and Keras). Using this information, we analyze the distribution of bug-introducing commits and the survival time of bugs.

This study addresses the following research questions:

- **RQ1:** Do ML-related code changes introduce more bugs than non-ML code changes?
- **RQ2:** Do ML-related bugs remain undetected longer than non-ML bugs?

The contributions of this paper are as follows:

- We provide empirical evidence on the distribution of bug-introducing commits across ML and non-ML code in open-source ML products.
- We conduct a survival analysis of bugs to compare the lifespan of ML-related and non-ML-related defects.
- We construct a dataset of bug-introducing commits in ML software repositories to support future research on ML software quality.

The remainder of this paper is organized as follows. Section II describes our research methodology. Section III presents the results of our empirical analysis. Section **??** shows the related work, Section IV explains the discussions and implications of our work. Section VI discusses threats to validity, and Section VII concludes the paper.

II. Methodology

To investigate the differential characteristics and evolutionary impact of defects in machine learning (ML) components versus traditional software code, we established an automated repository mining and analysis pipeline. Our approach is specifically designed to isolate mixed-domain repositories and trace the complete lifecycle of software defects within them from their initial introduction to their eventual resolution. The following subsections detail our multi-phase methodology, encompassing the rigorous selection of subject projects, the identification of corrective maintenance commits, the tracing of defect origins, and the automated classification of ML versus non-ML source code. An overview of the methodology is shown in Figure 1

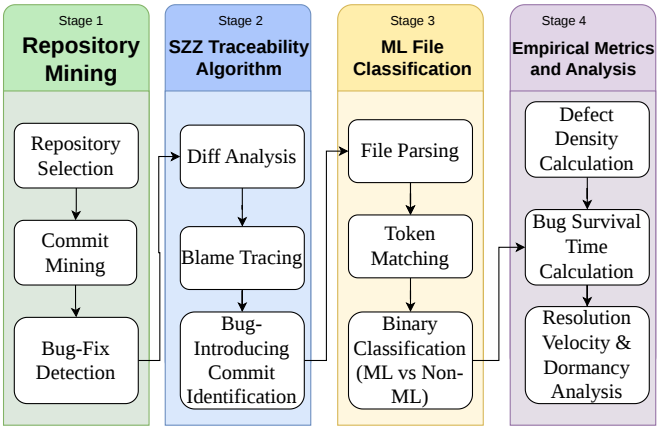

Fig. 1: Overview of Methodology

A. Project Selection

We used the dataset provided by Nahar et al. [4], which contains information on 262 open-source ML software products that are used as part of larger software systems. In order to filter the dataset and obtain mature software systems with a high evolutionary history, we used the following inclusion criteria:

- Community Adoption: The software had to contain at least 200 GitHub stars.
- Development History: The software had to contain at least 5,000 commits.
- Language Dominance: The software had to contain a high number of Python files.

Using the above inclusion criteria, we obtained a list of 13 high-quality software systems that we used as the basis of our dataset. These software systems represent complex software systems with high interactions between traditional software and ML components.

B. Data Collection and Filtering

We started with the cloning of the complete version control histories of some large-scale, open-source software systems known to incorporate not only conventional software architecture but also machine learning pipelines. To identify the corrective maintenance activities, we employed a keyword-based heuristic approach on the commit messages. We parsed the Git commit history to look for specific words like 'fix,' 'bug,' 'issue,' 'patch,' and 'resolves'. This approach is a well-established technique in empirical software engineering for identifying bug-fixing commits. [5]

C. Tracing Defect Origins (SZZ Algorithm)

Upon identifying the set of bug-fixing commits, we applied the SZZ algorithm [3] to locate the exact commit that introduced the defect. For each bug-fixing commit, we extracted the code churn (lines added and deleted) using the Git diff. We then utilized the git blame utility on the modified files to trace the deleted or altered lines back to their most recent prior modification. These prior modifications are flagged as the bug-introducing commits. This step allows us to establish a lifecycle for each defect, from introduction to resolution, calculating the bug survival time ($T_{survival}$) as: $T_{survival} = t_{fix} - t_{intro}$where t_{fix} and t_{intro} represent the committed timestamps of the fixing and introducing commits, respectively.

D. ML and Non-ML Component Classification

To evaluate our core research question, we needed to partition the codebase into ML and non-ML domains. We developed an automated classification script that analyzes the source code at the exact point of the HEAD commit. For each Python file within a repository, we extracted the raw file content and performed token matching against a predefined list of widely used machine learning and computer vision libraries, including TensorFlow, PyTorch, Scikit-learn, Hugging Face, and OpenCV as done in [6]–[8]. If a file contained explicit dependencies or references to these libraries, we classified it as an ML-related file. All other files were designated as non-ML files. This binary classification forms the foundation of our analysis, allowing us to comparatively analyze defect density, code churn, and bug survival duration across the two distinct domains to understand the impact of hidden technical debt [9]

III. Results

We present the findings of our empirical analysis across the 13 selected repositories. We structure our results by directly addressing our two core research questions.

A. RQ1 : Do ML-related code changes introduce more bugs than non-ML code changes?

In order to answer RQ1, we compared the architectural footprint of the ML components with their contribution to the overall number of bugs introduced. In total, we observed that the number of ML-specific python files was only 4.77% on average of the overall python codebase within the analyzed codebases. However, as

In order to rigorously quantify this discrepancy, we computed the defect density (bugs per file) for the two domains. Our results showed that the code for ML is more defect-prone compared to the code for regular software. To be more specific, the median defect density for the ML code is 2.73 bugs per file, while for the non-ML code, the corresponding figure is only 3.56 bugs per file (See Figure 3).

TABLE I: Overview of Subject Projects, Defect Proportions, and ML Risk Statuses.

Repository	Python Files	ML Files (%)	Total Bugs	ML Bugs (%)	ML Risk Status	Avg. Resolution
CellProfiler/CellProfiler	859	3.1 %	3,366	2.3 %	Nominal	Dormant
OpenBB-finance/OpenBBTerminal	1,268	1.0 %	2,509	2.0 %	High Risk	Standard
SteveDoyle2/pyNastran	1,533	0.7 %	1,560	0.9 %	Disproportionate	Dormant
electricitymap/electricitymap-contrib	286	4.5 %	1,904	1.8 %	Nominal	Dormant
freqtrade/freqtrade	468	9.8 %	6,130	3.8 %	Nominal	Standard
galaxyproject/galaxy	2,316	13.2 %	1,154	1.1 %	Nominal	Swift Fix
ilastik/ilastik	839	11.3 %	2,467	15.4 %	Disproportionate	Prolonged
mayan-edms/Mayan-EDMS	2,203	1.0 %	1,700	0.8 %	Nominal	Dormant
metabrainz/listenbrainz-server	610	0.8 %	3,459	0.2 %	Nominal	Dormant
nautechsystems/nautilus_trader	1,048	1.1 %	3,277	0.2 %	Nominal	Swift Fix
psychopy/psychopy	652	0.8 %	4,386	1.0 %	Disproportionate	Dormant
pupil-labs/pupil	303	12.9 %	1,549	25.3 %	High Risk	Prolonged

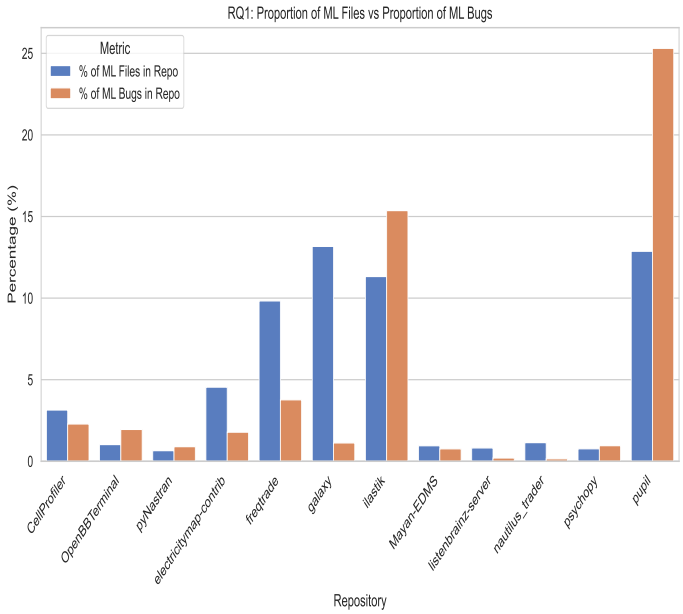

Fig. 2: Proportion Barchart of ML and non-ML bugs

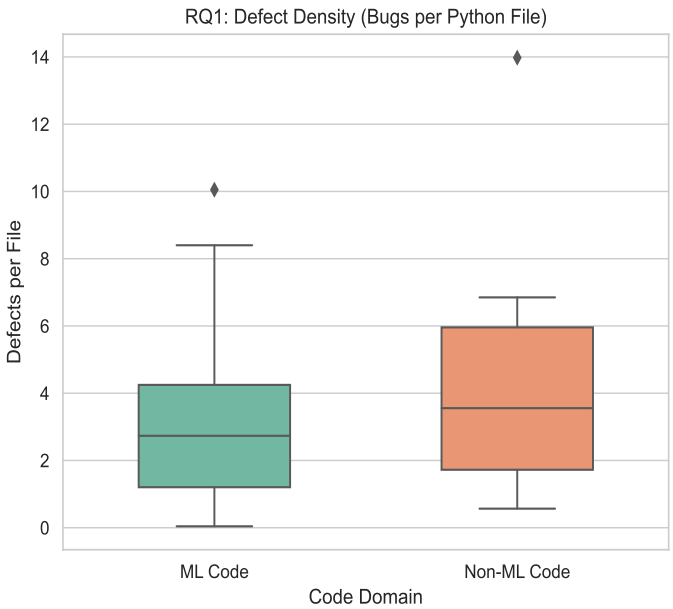

Fig. 3: Defect Density across ML and non-ML files

Furthermore, as detailed in Table I, the majority of the subject systems exhibited a "High Risk" or "Disproportionate" ML Risk Status, indicating that the integration of machine learning libraries inherently increases the likelihood of defect introduction compared to traditional software engineering tasks.

B. RQ2: Do ML-related bugs remain undetected longer than non-ML bugs?

Having established the defect density of ML components, we next investigated the timeframe required to detect and resolve these issues. We calculated the bug survival time where we calculated the duration in days between the bug-introducing commit and the subsequent fixing commit for all traced defects. Contrary to initial expectations, our findings indicate that ML defects exhibit a substantially shorter lifespan within the system. As shown in the log-scaled distribution in Figure 4, the median survival time for an ML bug was 41.34 days (IQR: 3.93 – 254.05 days). In contrast, standard non-ML bugs lingered significantly longer before resolution, with a median survival time of 80.92 days, nearly double that of their ML counterparts.

To understand the resolution velocity, we plotted the Cumulative Distribution Function (CDF) of bug lifespans (Figure 5). The survival curves demonstrate a faster resolution rate for ML defects in the early stages of the defect lifecycle. Within a standard 30-day window following introduction, 46% of ML-related bugs were detected and fixed. However, only 38.1% of standard software bugs were successfully resolved within that same timeframe. While ML defects are addressed more rapidly overall, a notable long tail still exists, with approximately 40% of ML defects falling into the "Prolonged" or "Dormant"

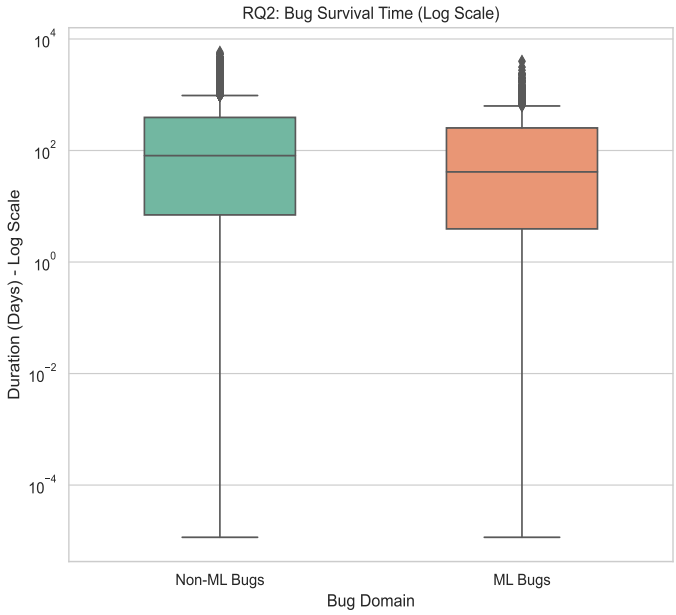

Fig. 4: Bug survival time between ML and non-ML bugs

resolution categories, surviving well beyond 90 days.

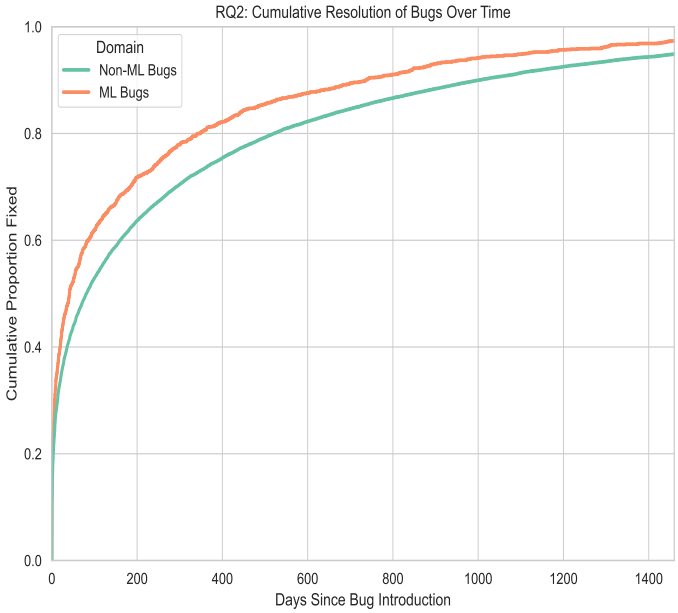

Fig. 5: Cumulative Resolution of bugs over time

This extended survival time suggests that ML bugs may be more "silent" or difficult to reproduce, allowing them to hide in the codebase longer than traditional software faults before triggering a recognizable failure.

IV. Discussion & Implications

Our empirical analysis of open-source ML software reveals a shocking pattern with respect to the survival of ML defects. Contrary to the intuition that ML defects take longer to be detected due to their data-dependent nature , we observe that ML defects are fixed much faster than non-ML defects, with a median survival time of 41.34 days compared to 80.92 days for non-ML defects. This indicates that although ML code is extremely defect-prone, the defects that do occur do so instantaneously and catastrophically. Defects such as tensor shape mismatches or faulty data pipelines cause the program to come to a complete stop.

However, a significant long tail is still present with 40% of the ML defects categorized as "Prolonged" or "Dormant" resolution types, lasting longer than 90 days. This indicates a dual nature of ML defect behavior: implementation defects crash soon after their introduction, but semantic or data-driven defects can linger in the codebase for a significantly longer time than conventional software defects before causing a failure.

Our findings indicate that defect prediction and reliability models developed for traditional software may not directly apply to ML-based systems. The 'fast crash-long tail' defect pattern points to the need to explore new defect prediction approaches tailored to ML-based systems that take into account data dependencies, pipeline complexity, and model lifecycle characteristics. In addition, the existence of long-lived ML defects underlines the need to explore technical debt accumulation in ML-based systems and to develop tools to automatically identify silent failures during the software lifecycle.

V. Related Work

Our study builds upon and extends several different streams of research, encompassing the classification of machine learning faults, the analysis of software quality and technical debt in ML systems, and the application of repository mining techniques.

A. Taxonomy and Characteristics of ML Bugs

With the increased prevalence of ML in software systems, researchers have attempted to identify the specific defects that can arise in these systems. Humbatova et al. suggested an extensive taxonomy for actual defects occurring in deep learning systems, where the importance of the distinction between ML-related defects and the entire system is emphasized [1]. In the similar context of ML-related defects, another research study was conducted by Zhang et al. to suggest an empirical study for actual ML-related bugs, where the root causes and symptoms of the actual defects occurring in the software system are determined [2]. Although the research study suggested the importance of the actual taxonomies for ML-related defects, the research study is an extension of the ideas suggested by the research study, where the survival time for ML-related defects is compared with regular defects.

B. Technical Debt and Code Quality in ML Systems

The amalgamation of the ML code with other software programs through different computational paradigms [8] often results in complexities related to the maintenance of the system. Sculley et al. proposed the idea of "hidden technical debt" for ML systems, with a focus on the complexities related to the maintenance of the system [9]. Similar to the above idea,

another research study by Van Oort et al. proposed an empirical study related to the quality of the code for ML systems [6], with a focus on the prevalence of the code smells for ML systems. In another research study, Siddik and Bezemer proposed an important research study related to the distinction between the quality of the code and the code smells for ML systems and non-ML systems [7]. Our research builds upon the above ideas by indicating the high density of defects for the ML files compared to the non-ML files.

C. Repository Mining and Software Evolution

Our approach is strongly rooted in the well-established empirical research methodologies that are a part of the field of software engineering. For the detection of the bug-introducing commits, we employ the highly successful SZZ algorithm, which was first proposed in the research work done by Sliwerski et al. [3]. For the restriction of the analysis to corrective maintenance activities, we employ a set of heuristics that are based on the commit history. This work was inspired by the research work done by Mockus and Votta [5]. The central contribution of this research work, the dataset as well as the architectural framework, are the results of the large-scale empirical research work done by Nahar et al. to present the bigger picture of the software beyond the ML models [4].

VI. Threats to Validity

Several threats to construct and internal validity exist in our automated mining pipeline. Firstly, the identification of bug fixing commits through keyword heuristics could result in false positives, and the subsequent application of the SZZ algorithm could incorrectly identify the cause of the defect as cosmetic instead of logical error. However, our approach avoids this problem by aggregating defect densities and median survival times instead of individual commits. Additionally, our classification approach uses the current imports of a file to determine the class of the file. Thus, a file classified as 'ML' could have not been an ML file when the bug was introduced.

In terms of external validity, it should be noted that this study is based on a pre-selected set of 13 open-source code repositories, curated from the data set by Nahar et al. [4]. While these are clearly mature, complex systems in which machine learning and traditional software code coexist, it should also be noted that defect lifespans and densities in ML systems developed in a corporate quality assurance context may differ from those found in this data set. It should also be noted that this study's inclusion criteria and architectural heuristics were specific to the Python ML ecosystem; thus, it is possible that defect characteristics and ML system evolutionary patterns in ML systems coded in languages such as C++ and Java may not perfectly match those found in this data set.

VII. Conclusion

We showed that while hybrid systems' ML parts were over-represented among defect-prone software parts, their resolution occurs almost twice as fast as non-ML defect resolution, despite 40% of them being dormant. In the context of the above-described emerging findings, our future work will aim to carry out detailed root cause analysis to distinguish between fast-crashing ML defects and long-term technical debt, as well as to explore the effect of structural coupling between domains on defect acceleration. Ultimately, by expanding our dataset to more languages and systems, we hope to apply the above-described findings to build ML-specific fault prediction models and CI tools.

REFERENCES

[1] N. Humbatova, G. Jahangirova, G. Bavota, and F. Ricca, "Taxonomy of real faults in deep learning systems," in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. IEEE, 2020, pp. 1110–1121.

[2] T. Zhang, J. Chen, J. Yang *et al.*, "An empirical study of real-world machine learning bugs," in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. IEEE, 2020, pp. 1116–1127.

[3] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" *ACM SIGSOFT Software Engineering Notes*, vol. 30, no. 4, pp. 1–5, 2005.

[4] N. Nahar, H. Zhang, G. Lewis, S. Zhou, and C. Kästner, "The product beyond the model: An empirical study of repositories of open-source ML products," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1540–1552.

[5] A. Mockus and L. G. Votta, "Identifying reasons for software changes using historic databases," in *Proceedings of the International Conference on Software Maintenance*. IEEE, 2000, pp. 120–130.

[6] B. Van Oort, L. Cruz, M. Aniche, and A. Van Deursen, "The prevalence of code smells in machine learning projects," in *2021 IEEE/ACM 1st Workshop on AI Engineering–Software Engineering for AI (WAIN)*. IEEE, 2021, pp. 1–8.

[7] M. S. Siddik and C.-P. Bezemer, "Do code quality and style issues differ across (non-) machine learning notebooks? yes!" in *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2023, pp. 72–83.

[8] V. Liermann, "Overview of machine learning and deep learning frameworks," in *The Digital Journey of Banking and Insurance, Volume III: Data Storage, Data Processing and Data Analysis*. Springer, 2021, pp. 187–224.

[9] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," *Advances in Neural Information Processing Systems*, vol. 28, 2015.